

Chapter 8 Tree

TREE DATA STRUCTURE

Prompt

What is a tree in data structures?

Answer

A **tree** is a **non-linear, hierarchical data structure** where:

- One element is at the top (called the **root**)
- Every element (except the root) has **one parent**
- Each element can have **zero or more children**

Unlike arrays or lists, trees do **not** store data sequentially.

Basic Tree Terminology

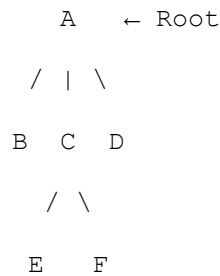
Prompt

Define root, parent, child, sibling, leaf, internal node.

Answer

Term	Meaning
Root	Topmost node with no parent
Parent	A node that has children
Child	Node directly below a parent
Sibling	Nodes with the same parent
Leaf (External node)	Node with no children
Internal node	Node with at least one child

♣ Example Diagram



- Root: A
- Parent of E & F: C
- Siblings: B, C, D
- Leaves: B, D, E, F
- Internal nodes: A, C

Prompt

Give the formal definition of a tree.

Answer

A tree **T** is a set of nodes such that:

1. If **T is not empty**, it has a **root** with no parent
2. Every other node has **exactly one parent**
3. Nodes form a **parent–child relationship**
4. A tree can be **empty**

Recursive Definition

A tree is:

- **Empty**, OR
- A root node with **zero or more subtrees**

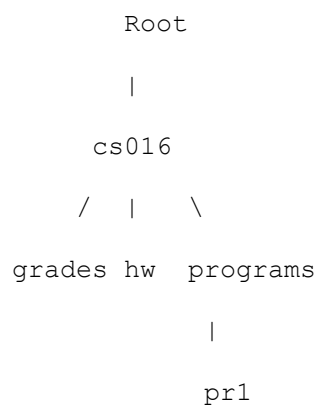
Prompt

What are ancestor, descendant, and subtree?

Answer

- **Ancestor:** A node on the path from the root to a node
- **Descendant:** Any node below a given node
- **Subtree:** A node and all its descendants

♣ Example



- Root is ancestor of all
- pr1 is descendant of cs016
- Subtree rooted at programs:

programs

|

pr1

Prompt

What is an edge and a path in a tree?

Answer

- **Edge:** Connection between parent and child
- **Path:** Sequence of nodes connected by edges

Example Path:

cs252 → projects → demos → market

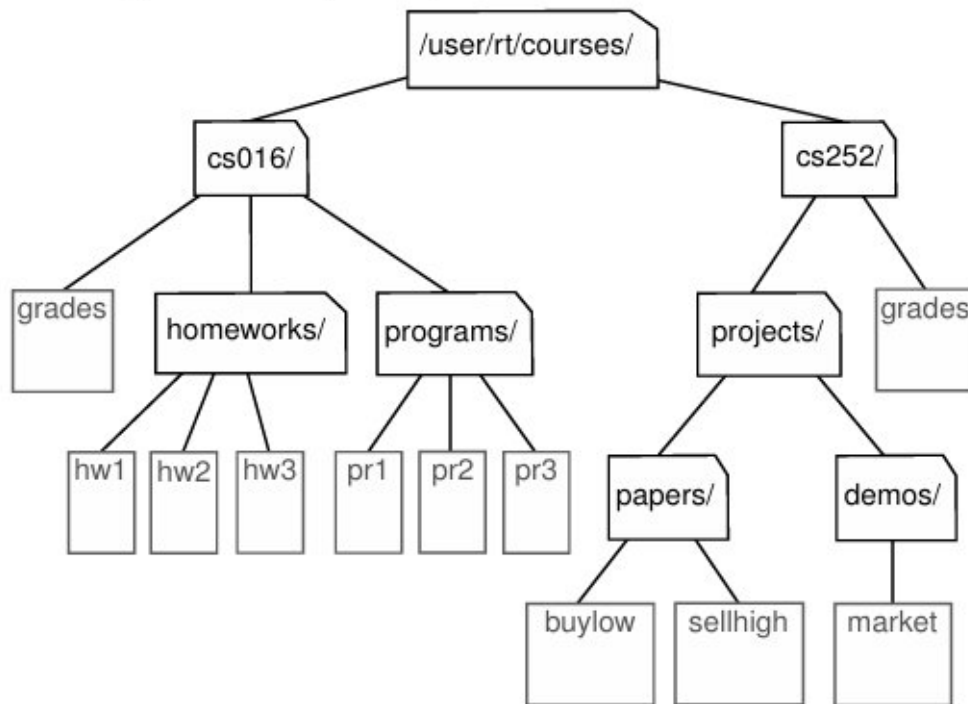


Figure 8.3: Tree representing a portion of a file system.

Prompt

What is the difference between internal and external nodes?

Answer

Type	Definition
Internal Node	Has at least one child
External Node (Leaf)	Has no children

■ File system example:

- Directories → internal nodes
- Files → leaf nodes

Prompt

Give real-life examples of trees.

Answer

Example	Root	Nodes
File System	/	folders & files
Company Structure	CEO	departments
Book Structure	Book	chapters, sections
Programming	Base class	derived classes

Prompt

What is an ordered tree?

Answer

An **ordered tree** is a tree where the **order of children matters**.

■ Ordered Tree Example (Book)

Book

```
├─ Chapter 1
|   ├── Section 1.1
|   └─ Section 1.2
├─ Chapter 2
└─ Chapter 3
```

Changing chapter order changes meaning → **ordered tree**

✗ Unordered Tree Example

- Company hierarchy
- Class inheritance
- Organizational charts

Order of children **does not matter**

Prompt

How is a tree used in programming?

Answer

In **object-oriented programming**, class inheritance forms a tree.

Python Example:

`object`

|

`BaseException`

|

`Exception`

|

`ArithmeticError`

|

`ZeroDivisionError`

- `object` is the root
- All classes inherit directly or indirectly from it

THE TREE ABSTRACT DATA TYPE (ADT)

Prompt

What is the Tree Abstract Data Type?

Answer

The **Tree ADT** defines **what operations a tree should support**, not **how it is implemented**.

It describes:

- How tree elements are accessed
- How parent–child relationships are navigated
- What operations are allowed

☞ It focuses on **behavior**, not storage.

Prompt

What is a Position in the Tree ADT?

Answer

A **Position** is an **abstraction of a node** in a tree.

- It represents **where an element is stored**
- Users interact with **positions**, not nodes directly
- Each position stores **one element**

Method supported:

`p.element()`

Returns the element stored at position `p`.

♣ Diagram (Position Concept)

```
    [ A ] ← Position p1
    /      \
  [ B ]     [ C ] ← Positions p2, p3
```

- `p1.element() → A`
- `p2.element() → B`

Prompt

Why does the Tree ADT use positions?

Answer

Using positions:

- Hides internal implementation
- Improves **data abstraction**
- Prevents users from corrupting structure
- Allows multiple tree implementations to share the same interface

Prompt

List and explain Tree ADT accessor methods.

Answer

Method	Description
<code>T.root()</code>	Returns root position
<code>T.is_root(p)</code>	Checks if <code>p</code> is root
<code>T.parent(p)</code>	Returns parent of <code>p</code>
<code>T.num_children(p)</code>	Number of children
<code>T.children(p)</code>	Iterates over children
<code>T.is_leaf(p)</code>	Checks if <code>p</code> has no children
<code>len(T)</code>	Total nodes
<code>T.is_empty()</code>	Checks if tree is empty
<code>T.positions()</code>	Iterates all positions
<code>iter(T)</code>	Iterates all elements

Prompt

What happens if an invalid position is used?

Answer

If a method receives an **invalid position**, it must raise:

`ValueError`

This ensures:

- Tree integrity
- Safe navigation
- No misuse of positions

Prompt

How does Tree ADT handle ordered trees?

Answer

- In an **ordered tree**, `T.children(p)` returns children in **fixed order**
- In an **unordered tree**, order has **no meaning**

If p is a leaf:

- `T.children(p) → empty iteration`

If T is empty:

- `T.positions() → empty`
- `iter(T) → empty`

Prompt

Why doesn't Tree ADT define insert or delete operations?

Answer

Because:

- Update methods depend on **tree representation**
- Different trees (binary, general, array-based, linked) update differently
- Cleaner design to define updates in **concrete implementations**

Prompt

What is an abstract base class?

Answer

An **abstract base class**:

- Defines a **common interface**
- Contains **abstract methods** (not implemented)
- May contain **concrete methods**
- Cannot be instantiated directly

Prompt

Why is a Tree abstract base class used?

Answer

It:

- Defines the Tree ADT formally
- Enables **code reuse**
- Enforces consistent behavior across implementations
- Allows multiple tree types to share logic

Prompt

What is the nested Position class in Tree?

Answer

The `Position` class:

- Represents a location in the tree
- Is **abstract**
- Must be implemented by subclasses

Methods:

`element()`

`__eq__()`

`__ne__()`

Prompt

Which Tree methods are abstract and why?

Answer

Abstract methods (must be implemented by subclasses):

Method	Purpose
<code>root()</code>	Access root
<code>parent(p)</code>	Access parent
<code>num_children(p)</code>	Count children

<code>children(p)</code>	Iterate children
<code>__len__()</code>	Size of tree

They depend on **how the tree is stored**.

Prompt

Which methods are concrete in Tree class?

Answer

`is_root(p)`

`is_leaf(p)`

`is_empty()`

Why concrete?

They can be written **using abstract methods**, so no need to rewrite them in subclasses.

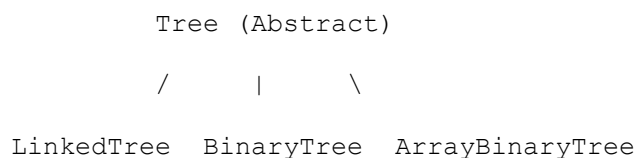
Prompt

How does Tree abstract base class promote code reuse?

Answer

- Concrete methods are inherited automatically
- Subclasses only implement core storage logic
- Avoids duplicate code across implementations

♣ Tree Class Design Diagram



- `Tree` defines interface
- Subclasses implement storage

Prompt

Why can't we create an object of Tree class?

Answer

Because:

- It has abstract methods
- It has no internal data structure
- It is incomplete by design

BINARY TREES

Prompt

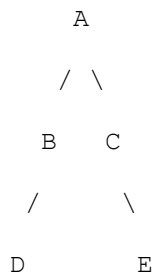
What is a binary tree?

Answer

A **binary tree** is an **ordered tree** with these properties:

1. Each node has **at most two children**
2. Children are labeled as **left child** and **right child**
3. The **left child always comes before** the right child

♣ Basic Binary Tree Diagram



- A has two children
- B has one left child
- Order matters: left $\neq$ right

Prompt

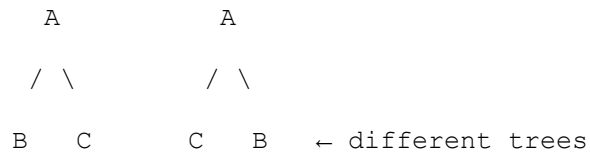
Why is a binary tree considered an ordered tree?

Answer

Because:

- Each child has a **distinct position** (left or right)
- Swapping left and right children **changes the tree**

Example:



Prompt

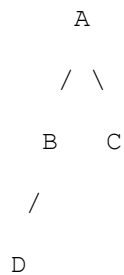
What are left and right subtrees?

Answer

For any node v :

- The subtree rooted at its **left child** → **left subtree**
- The subtree rooted at its **right child** → **right subtree**

♣ Example



- Left subtree of A:



- Right subtree of A:

C

Prompt

What is a proper binary tree?

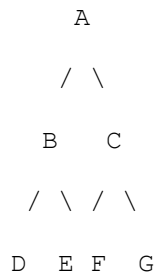
Answer

A **proper binary tree** is a binary tree where:

- Every node has **either 0 or 2 children**
- No node has exactly one child

Also called a **full binary tree**

✓ Proper Binary Tree



✗ Improper Binary Tree



- B has only one child → **improper**

Prompt

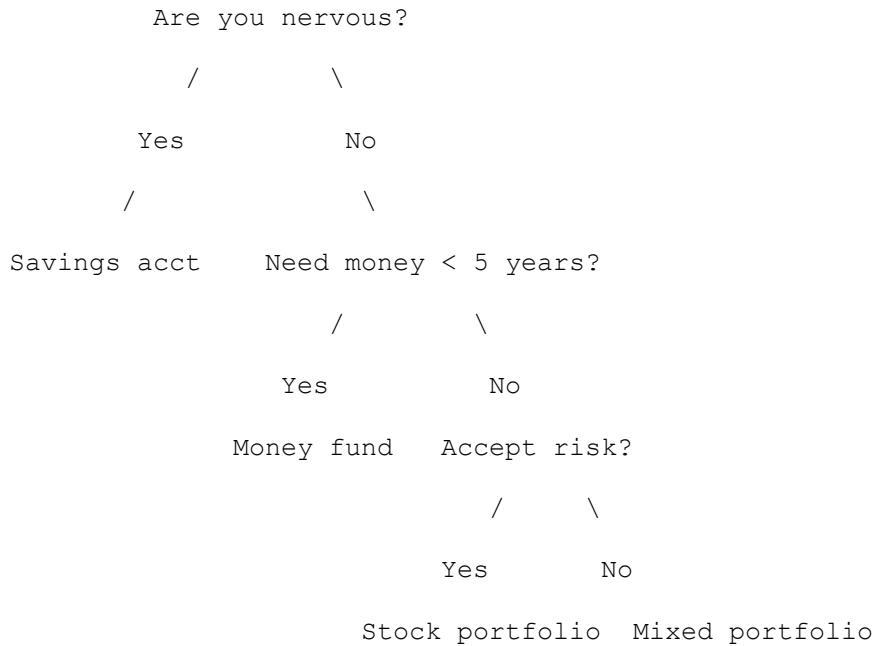
What is a decision tree?

Answer

A **decision tree** is a **proper binary tree** used to represent:

- A series of **yes/no questions**
- Each internal node → a question
- Each edge → yes or no
- Each leaf → final decision

♣ Decision Tree Diagram



Prompt

Why is a decision tree a proper binary tree?

Answer

Because:

- Every question has **exactly two answers** (Yes/No)
- Internal nodes have **two children**
- Leaves have **no children**

Prompt

What is an expression tree?

Answer

An **expression tree** is a binary tree where:

- **Leaves** → operands (numbers/variables)

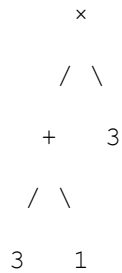
- **Internal nodes** $\rightarrow$ operators (+, -, $\times$, /)

♣ Example Expression Tree

Expression:

$$(3 + 1) \times 3$$

Tree:



Prompt

How is the value of an expression tree computed?

Answer

- If node is a **leaf** $\rightarrow$ return value
- If node is **internal** $\rightarrow$ apply operator to children

Example:

$$(3 + 1) \times 3 = 4 \times 3 = 12$$

PROPERTIES OF BINARY TREES

Prompt

What is a level in a binary tree?

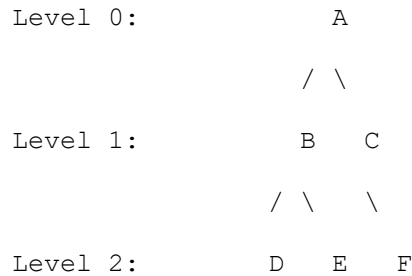
Answer

- **Level d** of a binary tree is the set of all nodes at **depth d**
- Depth = number of edges from the root

Example:

- Level 0 → root
- Level 1 → children of root
- Level 2 → grandchildren of root

♣ Diagram: Levels in a Binary Tree



Prompt

What is the maximum number of nodes at level d?

Answer

In a binary tree:

- Level 0 → at most **1** node
- Level 1 → at most **2** nodes
- Level 2 → at most **4** nodes
- Level 3 → at most **8** nodes

🔑 General Formula

$$\text{Max nodes at level } d = 2^d$$

📊 Table

Level (d)	Max Nodes
0	1
1	2
2	4
3	8
d	2^d

Prompt

What is the height of a binary tree?

Answer

- Height (**h**) = maximum depth of any node
- Root alone $\rightarrow$ height = 0

Prompt

What is the relationship between height and number of nodes?

Answer (Proposition 8.8)

For a **nonempty binary tree**:

$$h + 1 \leq n \leq 2^{h+1} - 1$$

Meaning:

- **Minimum nodes** $\rightarrow$ tree is skewed
- **Maximum nodes** $\rightarrow$ tree is perfect

♣ Example

Minimum nodes (skewed tree)

A

\

B

\

C

Height = 2, Nodes = 3 $\rightarrow h+1 = 3$ ✓

Maximum nodes (perfect tree)

A

/ \

B C

/ \ / \

D E F G

Height = 2, Nodes = 7 $\rightarrow 2^3 - 1 = 7$ ✓

Prompt

What is the range of external nodes in a binary tree?

Answer

$$1 \leq n_E \leq 2^h$$

Where:

- n_E = number of leaf nodes
- h = height

Prompt

What is the range of internal nodes in a binary tree?

Answer

$$h \leq n_I \leq 2^h - 1$$

Where:

- n_I = number of internal nodes

Prompt

How is height related to number of nodes?

Answer

$$\log_2(n + 1) - 1 \leq h \leq n - 1$$

- **Minimum height** → balanced tree
- **Maximum height** → skewed tree

Prompt

What is a proper binary tree?

Answer

A **proper (full) binary tree** is a binary tree where:

- Every internal node has **exactly two children**
- No node has only one child

♣ Proper vs Improper Tree

Proper



Improper



Prompt

What special properties hold for proper binary trees?

Answer

Property	Formula
Total nodes	$2^{h+1} \leq n \leq 2^{h+1} - 1$
External nodes	$h + 1 \leq n_E \leq 2^h$
Internal nodes	$h \leq n_I \leq 2^h - 1$
Height	$\log(n + 1) - 1 \leq h \leq (n - 1) / 2$

Prompt

What is the relationship between internal and external nodes in a proper binary tree?

Answer (Proposition 8.9)

$$n_E = n_I + 1$$

♣ Very important exam formula

Prompt

Explain why external nodes = internal nodes + 1.

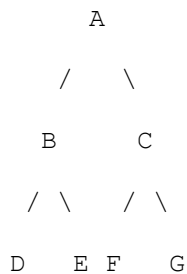
Answer (Intuition)

- Each internal node contributes **2 children**
- One child becomes an internal node
- The extra child becomes a leaf

This imbalance results in:

$$\text{Leaves} = \text{Internal nodes} + 1$$

♣ Example



- Internal nodes = A, B, C $\rightarrow 3$
 - External nodes = D, E, F, G $\rightarrow 4$
- ✓ $4 = 3 + 1$

Prompt

Explain the removal process used to justify Proposition 8.9.

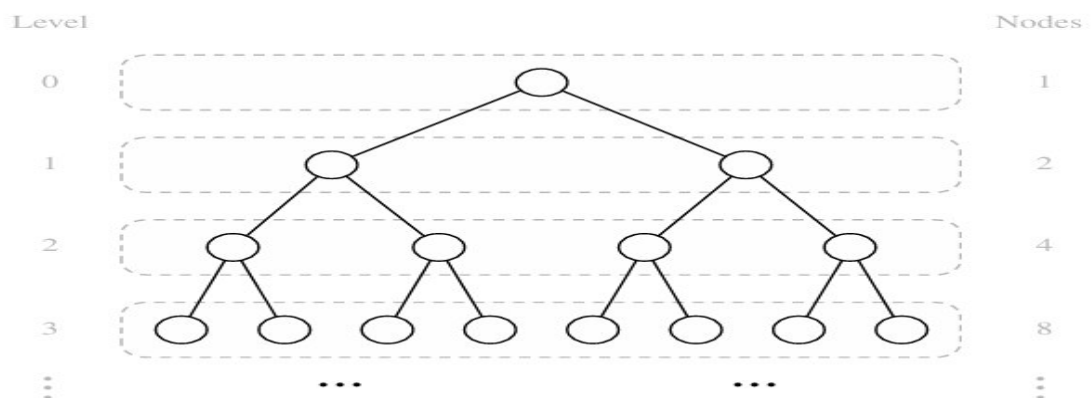


Figure 8.9: Maximum number of nodes in the levels of a binary tree.

Answer

- Remove one leaf and its parent
- Place leaf $\rightarrow$ external pile

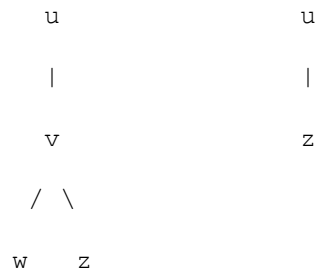
- Place parent → internal pile
- Reconnect remaining child

Eventually:

- One final node remains → external
- External pile has **one extra node**

♣ Removal Illustration

Before: After:



Prompt

Does $n_E = n_I + 1$ apply to all trees?

Answer

✗ No

✓ It applies **only to proper binary trees**

It does **not** hold for:

- Improper binary trees
- Non-binary trees

LINKED STRUCTURE FOR BINARY TREES

Prompt

What is a linked structure representation of a binary tree?

Answer

A **linked structure** represents a binary tree using **nodes connected by references (links)** instead of arrays.

Each node stores:

- The **element**
- A reference to its **parent**
- A reference to its **left child**
- A reference to its **right child**

The tree object itself stores:

- A reference to the **root**
- The **size** (number of nodes)

Prompt

Why is a linked structure a natural choice for binary trees?

Answer

Because:

- Binary trees are **nonlinear**
- Nodes have **variable positions**
- Linked structures allow:
 - Easy insertion/deletion
 - No wasted memory (unlike arrays)
 - Natural parent–child relationships

Prompt

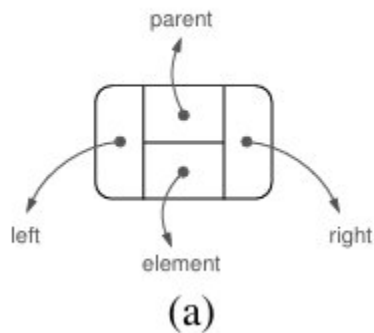
What information does a binary tree node store?

Answer

Each node stores **four fields**:

element | parent | left | right

♣ Diagram: Single Node (Figure 8.11a)



- This node could be the **root**
- Parent is `None`
- No children yet

Prompt

How are parent and child relationships represented?

Answer

- Each node keeps a reference to its **parent**
- Each node keeps references to its **left** and **right** children
- Missing links are stored as `None`

♣ Example

```
A
 / \
B   C
```

Node relationships:

- `A.parent` → `None`
- `A.left` → `B`
- `A.right` → `C`
- `B.parent` → `A`
- `C.parent` → `A`

Prompt

What does the binary tree object store?

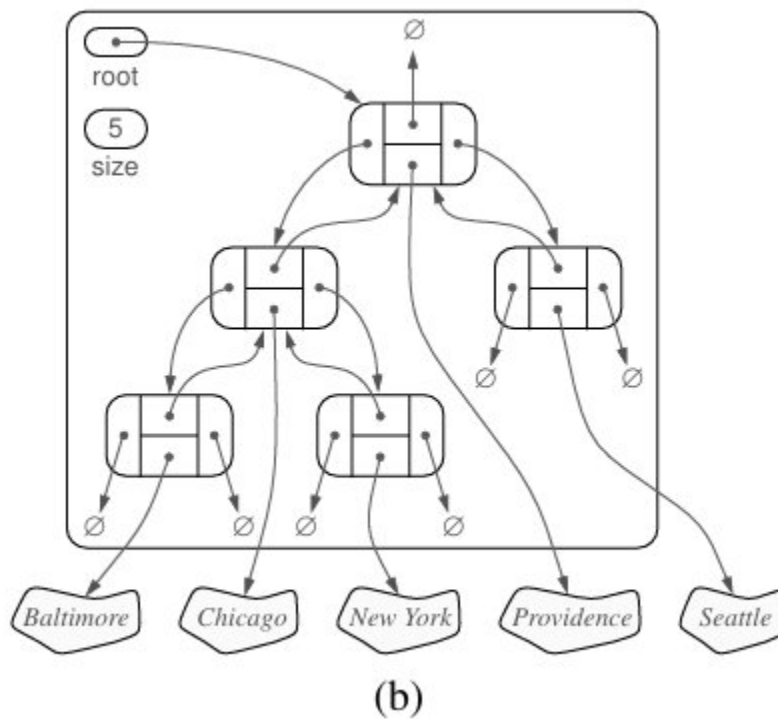
Answer

The tree object stores:

1. **root** → reference to root node
2. **size** → total number of nodes

Prompt

Explain the linked representation shown in Figure 8.11b.



Answer

Each node is connected via pointers forming a tree structure.

Prompt

How are missing children represented?

Answer

- If a node has **no left child**, `left = None`

- If a node has **no right child**, `right = None`
- If a node is root, `parent = None`

Prompt

What is the role of the size variable?

Answer

- `size` tracks the **number of nodes**
- Updated during insertion or deletion
- Enables `len(T)` in **O(1)** time

Prompt

What are the advantages of linked binary trees?

Answer

- ✓ Dynamic size
- ✓ Easy navigation (parent, left, right)
- ✓ No wasted memory
- ✓ Clean abstraction with Position objects

Prompt

What are the disadvantages of linked representation?

Answer

- ✗ Extra memory for pointers
- ✗ Slightly slower than arrays for complete trees
- ✗ More complex to implement

Prompt

How does linked structure fit the Tree ADT?

Answer

- Each node corresponds to a **Position**
- Tree methods (`parent`, `children`, etc.) return positions
- Internal node structure is **hidden from the user**

ARRAY-BASED REPRESENTATION OF BINARY TREES

Prompt

What is an array-based representation of a binary tree?

Answer

- Instead of linking nodes with pointers, we **store tree elements in an array (or Python list)**.
- The **position of a node** is represented by an **index in the array**.
- The array index is determined by the **level numbering function** $f(p)$.

Prompt

How are nodes numbered in a binary tree array?

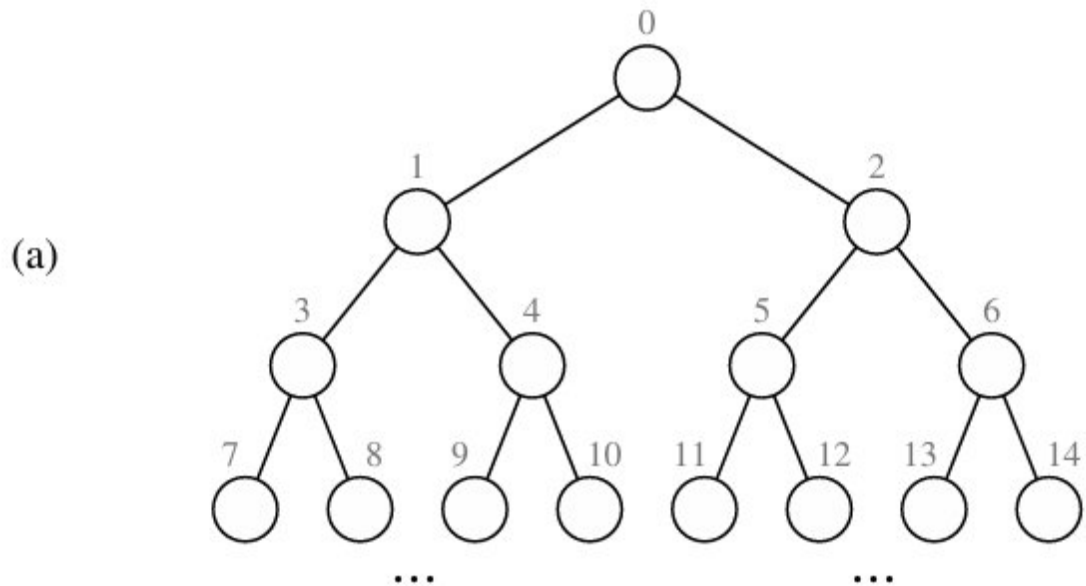
Answer

Let $f(p)$ be the index of position p :

1. **Root node** $\rightarrow f(\text{root}) = 0$
2. **Left child of q** $\rightarrow f(p) = 2f(q) + 1$
3. **Right child of q** $\rightarrow f(p) = 2f(q) + 2$

This is called **level numbering** — nodes on the same level are numbered left-to-right.

♣ Diagram: General Level Numbering



- Root $\rightarrow$ index 0
- Left child of 0 $\rightarrow 2*0+1 = 1$
- Right child of 0 $\rightarrow 2*0+2 = 2$
- Left child of 1 $\rightarrow 2*1+1 = 3$
- Right child of 1 $\rightarrow 2*1+2 = 4$

Example: Expression Tree Numbering

```

      /
     / \
    ×   +
   / \ / \
  +  3 4 2

```

Node	Index
/	0
×	1
+	2
+	3
3	4
4	5

2	6
---	---

Prompt

How can we compute parent and child positions using the array index?

Answer

Let $i = f(p)$ be the index of node p :

1. **Left child** $\rightarrow 2i + 1$
2. **Right child** $\rightarrow 2i + 2$
3. **Parent** $\rightarrow \lfloor (i-1)/2 \rfloor$

No pointers needed — just arithmetic!

♣ Diagram: Parent/Child Indices

Index:	0	1	2	3	4	5	6
Node:	/	×	+	+	3	4	2
Left:	1	3	5				
Right:	2	4	6				
Parent:	-	0	0	1	1	2	2

Prompt

Show an example of a binary tree as an array.

Answer

Binary Tree:

```

      /
     / \
    ×   +
   / \ / \
  +  3 4  2

```

Array representation (index $\rightarrow$ node):

Index	Node
0	/
1	×
2	+
3	+
4	3
5	4
6	2

Array = ['/', '×', '+', '+', 3, 4, 2]

Prompt

What are the advantages of array-based binary trees?

Answer

1. **Positions can be represented by integers** → simple arithmetic for navigation
2. **Root, parent, left, right methods** → $O(1)$ using formulas
3. Efficient for **complete binary trees** (e.g., heaps)

What is Tree Traversal?

Prompt

What is a tree traversal?

Answer

- **Traversal** is a systematic way to **visit every node/position** in a tree exactly once.
- The “visit” can perform any action:
 - Counting nodes
 - Printing elements
 - Computing a sum or expression
 - Building a list

Prompt

What are the main types of tree traversal for general trees?

Answer

For **general trees** (not just binary trees), common traversals are:

1. **Preorder Traversal** – Visit **root first**, then children recursively

2. Postorder Traversal – Visit **children first**, then root

For **binary trees**, there is also **inorder traversal**, which is specific to binary trees.

Prompt

What is a preorder traversal?

Answer

- Visit the **root node first**
- Then recursively traverse each subtree of its children (from left to right if ordered)

Pseudo-code

Algorithm preorder(T, p):

 visit(p) # perform some action on node p

 for each child c in $T.children(p)$:

 preorder(T, c) # recursively traverse subtree

♣ Example: Preorder Traversal

Tree (ordered):

```

    Paper
  /   |   \
Title Abstract $1
    /  \
  $1.1 $1.2
```

Preorder Visit Order:

Paper → Title → Abstract → \$1 → \$1.1 → \$1.2

♣ Diagram: Preorder Traversal

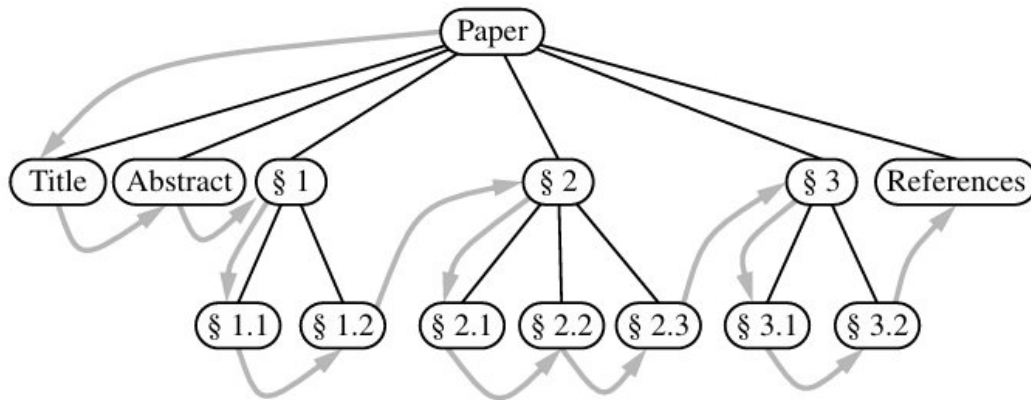


Figure 8.15: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

Prompt

What is a postorder traversal?

Answer

- Recursively visit **all children first**
- Then **visit the root node last**

Pseudo-code

```
Algorithm postorder(T, p):  
    for each child c in T.children(p):  
        postorder(T, c)  
    visit(p) # perform action after children
```

♣ Example: Postorder Traversal

Tree (same as above):

Postorder Visit Order:

Title → §1.1 → §1.2 → Abstract → §1 → Paper

🌲 Diagram: Postorder Traversal

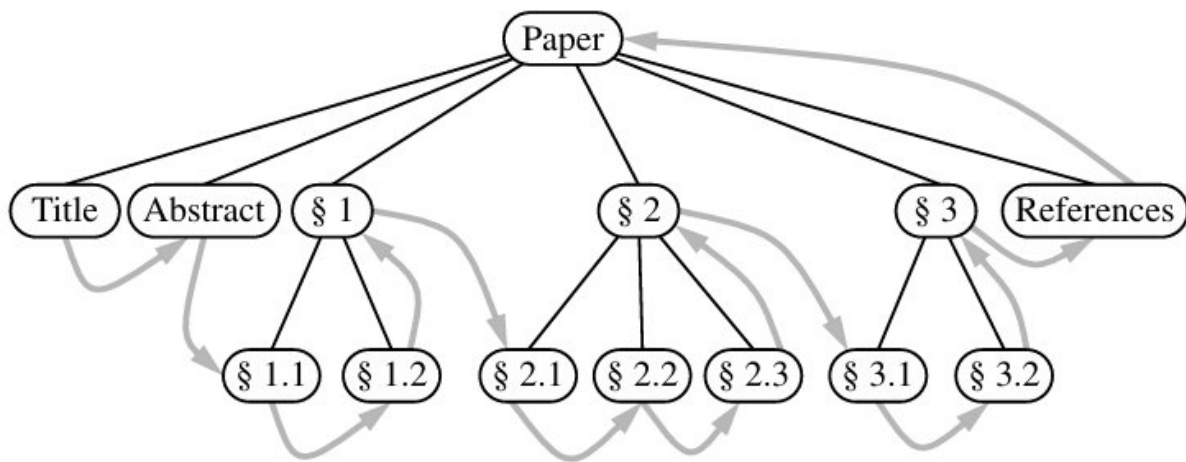


Figure 8.16: Postorder traversal of the ordered tree of Figure 8.15.

Tip: Think “children first, root last” for postorder.

Key Differences: Preorder vs Postorder

Feature	Preorder	Postorder
Visit root?	First	Last
Visit children?	After root	Before root
Use case	Copy tree, serialize	Delete nodes, evaluate expressions
Example order (Paper tree)	Paper, Title, Abstract, §1, §1.1, §1.2	Title, §1.1, §1.2, Abstract, §1, Paper

Visual Intuition

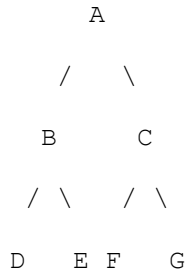
Preorder = “Top-down”

- Start at root → then children

Postorder = “Bottom-up”

- Start at leaves → then root

Tree Example:



Preorder: A B D E C F G

Postorder: D E B F G C A

Prompt

What is the time complexity of traversals?

Answer

- Visit each node exactly once
- Time complexity = $O(n)$
- Space complexity:
 - Recursive call stack = $O(h)$, where h = height of tree
 - Iterative traversal using stack $\rightarrow$ also $O(h)$

Prompt

What about inorder traversal?

Answer

- Specific to **binary trees**
- Visit **left subtree** $\rightarrow$ **root** $\rightarrow$ **right subtree**
- Applications:
 - Getting **sorted elements** from a **binary search tree (BST)**

Euler Tour Traversal of a Binary Tree

Prompt

What is the Euler Tour traversal of a binary tree?

Answer

- **Euler Tour** is a **generalized traversal** of a tree that visits **every node three times**:
 1. **Previsit** – before visiting any children
 2. **Invisit** – after visiting left child but before visiting right child (for binary trees)
 3. **Postvisit** – after visiting all children
- For **binary trees**, this traversal can **simulate preorder, inorder, or postorder** by overriding these hooks.

Prompt

What are the hooks in a Binary Euler Tour traversal?

Answer

Binary Euler Tour provides **three hooks**:

Hook	When Called	Purpose
<code>hook_previsit</code>	Before visiting any children	Simulates preorder
<code>hook_invisit</code>	After left child, before right child	Simulates inorder
<code>hook_postvisit</code>	After visiting both children	Simulates postorder

Hooks are **methods that can be overridden** in subclasses for custom behavior.

Prompt

How does the Binary Euler Tour algorithm work?

Answer

1. Initialize `results = [None, None]` for left and right subtree results
2. Call `hook_previsit(p, d, path)` for root node `p`
3. Recursively traverse **left child** (if exists), store result in `results[0]`
4. Call `hook_invisit(p, d, path)` after left subtree
5. Recursively traverse **right child** (if exists), store result in `results[1]`
6. Call `hook_postvisit(p, d, path, results)` after visiting both children
7. Return `answer`

Pseudo-code Simplified

```
def tour(self, p, d, path):  
    results = [None, None]  
    self.hook_previsit(p, d, path)  
  
    if self.tree.left(p) is not None:  
        path.append(0)  
        results[0] = self.tour(self.tree.left(p), d+1, path)  
        path.pop()  
  
    self.hook_invisit(p, d, path)  
  
    if self.tree.right(p) is not None:  
        path.append(1)  
        results[1] = self.tour(self.tree.right(p), d+1, path)  
        path.pop()  
  
    answer = self.hook_postvisit(p, d, path, results)  
    return answer
```

d = depth of node, path = sequence of left/right decisions

Relationship to Preorder, Inorder, Postorder

Traversal	Euler Hook to Override
Preorder	hook_previsit
Inorder	hook_invisit
Postorder	hook_postvisit

You can **simulate any traversal** by only implementing the corresponding hook.

Visualization of Binary Euler Tour

Prompt

How does the Euler Tour help compute a binary tree layout?

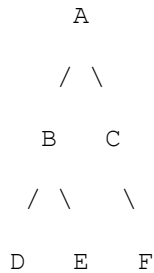
Answer

- Assign coordinates (x, y) to each node for drawing:
 1. $x(p)$ = number of **invisits performed before p** (horizontal position)
 2. $y(p)$ = **depth** of p (vertical position)
- Convention:
 - x increases **left** $\rightarrow$ **right**
 - y increases **top** $\rightarrow$ **bottom**

This gives a clear graphical layout where nodes are spaced according to an **inorder traversal**.

♣ Diagram: Binary Euler Tour Layout

Consider tree:



Inorder numbering (Euler Tour invisits):

1. $D \rightarrow x=0, y=2$
2. $B \rightarrow x=1, y=1$
3. $E \rightarrow x=2, y=2$
4. $A \rightarrow x=3, y=0$
5. $C \rightarrow x=4, y=1$
6. $F \rightarrow x=5, y=2$

$y=0$: $A(x=3)$

$y=1$: $B(x=1)$ $C(x=4)$

$y=2$: $D(x=0)$ $E(x=2)$ $F(x=5)$

This is exactly how `BinaryLayout` subclass uses `BinaryEulerTour` to compute coordinates.